

UPL — Universal Prompt Language

- **Version:** 1.0-rc.2
 - **Status:** Official Standard Specification
 - **File Extension:** `.txt` or `.upl`
-

1. Overview

The **Universal Prompt Language (UPL)** is a human-readable language for authoring parameterized self-contained prompt templates. A UPL file declares a set of typed input variables, their defaults and options, plus a prompt body containing placeholders, conditional expressions, and `for` loops. At render time, the engine substitutes variable values, evaluates conditionals with runtime type checking, and expands loops to produce the final prompt string.

UPL is designed to be:

- **Human-readable** — plain text, easy to author and review.
 - **Self-contained** — the upl prompt defined the complete list of parameters and the prompt body and its rendering logic
 - **Strictly typed** — every variable has an explicit type, validated at parse and render time.
 - **Safe** — only the `[[[` and `{{{` delimiters trigger expansion; unmatched blocks are ignored, so code snippets containing similar-looking sequences are not misinterpreted.
 - **Composable** — supports nested objects, lists of objects, reusable object shapes (`object_shape`), shape reuse via `type: <object_shape_name>`, and recursive structures.
-

2. File Format

A UPL file is a plain text file with the `.txt` or `.upl` extension. No other extension is permitted. It is divided into two or three sections separated by a line containing exactly `--`:

```
[--; optional]
<metadata section>
--
<prompt body section>
[--; optional, marks end of body]
```

- The **metadata section** is a YAML-like block declaring the prompt's identity and its input variables.
- The **prompt body** is free text that may contain placeholders, conditionals, and loops.
- A leading `--` line is **optional**: a file may begin directly with its first metadata key. A trailing `--` is also optional and conventionally used to mark the end of the body. Sections are separated by a line containing exactly `--`.

The `name` metadata field (\$2.1) **MUST** be equal to the file's base name (the file name with its `.txt` or `.upl` extension stripped). A single trailing `.prompt` segment — as produced by legacy tooling — is also stripped before the comparison, so both `my_prompt.txt` and `my_prompt.prompt.txt` resolve to the base name `my_prompt`. Any other suffix is not stripped. A mismatch between the `name` field and the file name is a parse/load error.

All values are **case-sensitive**. Variable and field *declarations* in `params` are **lowercase** identifiers, while every *reference* to a variable in the prompt body (placeholders, loop bindings, ternary branches, condition variables) **MUST** be **uppercase** and resolves case-insensitively against the declared name (see §4.1). Indentation uses **spaces only** (tabs are not permitted).

2.1 Metadata Fields

Field	Required	Description
<code>name</code>	Yes	Unique identifier of the prompt. MUST match the file's base name (see §2). Only lowercase alphanumeric (UTF-8) characters and underscores (<code>_</code>) are allowed; uppercase letters, hyphens, dots, and other punctuation are not permitted. An empty value is invalid.
<code>title</code>	No	Human-readable title.
<code>desc</code>	No	Optional description of the prompt's purpose.
<code>source</code>	No	Provenance field, <code><host>/<username>/<prompt_name></code> , injected automatically when a prompt is pulled from a UPL repository. Authoring tools SHOULD NOT set it by hand; it is informational and does not affect parsing or rendering.
<code>params</code>	Yes	Map of variable declarations (see §3).

2.2 Example Skeleton

```
--
name: my_prompt
title: My Prompt
desc: An example prompt
params:
  username:
    type: string
    desc: The user's name
    def: "guest"
--
Hello, [[[USERNAME]]]!
--
```

3. Variable Definitions

Variables are declared under the `params` block. Each variable has the following fields:

Field	Required	Applies to	Description
<code>type</code>	Yes	All	One of the types listed in §3.1.
<code>desc</code>	No	All	Optional human-readable description.
<code>def</code>	No	All	Default value used when no value is supplied. For <code>object_shape</code> , the declared <code>def</code> /field defaults are applied at every site that references the <code>object_shape</code> .
<code>opts</code>	No	<code>option_single</code> , <code>option_multi</code>	List of allowed options. Each entry must match the variable's <code>etype</code> (§3.3).
<code>etype</code>	Conditional	<code>list</code> , <code>option_single</code> , <code>option_multi</code>	Element type: a built-in type name (§3.1) — including the inline <code>object</code> (the variable then declares its element shape via its own <code>ofields</code>) — or the name of a declared <code>object_shape</code> variable (§3.4). Not allowed on <code>object</code> / <code>object_shape</code> (an object's shape is described by <code>ofields</code>). For <code>option_single</code> it is optional and defaults to <code>string</code> ; for <code>option_multi</code> it is required . The allowed

Field	Required	Applies to	Description
			etypes for <code>option_single</code> / <code>option_multi</code> are <code>string</code> , <code>long_string</code> , <code>number</code> , the inline <code>object</code> , and a referenced <code>object_shape</code> (§3.4). <code>boolean</code> , <code>list</code> , <code>option_single</code> , <code>option_multi</code> , and <code>object_shape</code> (the literal type name) are not valid option etypes. The allowed etypes for <code>list</code> are <code>string</code> , <code>long_string</code> , <code>number</code> , <code>boolean</code> , the inline <code>object</code> , and a referenced <code>object_shape</code> (§3.4); <code>list</code> , <code>option_single</code> , <code>option_multi</code> , and <code>object_shape</code> (the literal type name) are not valid list etypes.
<code>ofields</code>	Conditional	<code>object</code> , <code>object_shape</code>	Map of object field definitions (recursive). Required on <code>object_shape</code> ; on <code>object</code> either <code>ofields</code> (inline) or <code>type: <object_shape_name></code> (§3.4.2) must be present, but not both.
<code>label</code>	No	<code>option_single</code> , <code>option_multi</code>	Required when <code>etype</code> is a referenced <code>object_shape</code> : the field name (declared on that <code>object_shape</code>) whose value is shown as the menu label for each option. Ignored for scalar etypes.

`def` is **optional** for every type. When `def` is omitted (and no value is supplied interactively or programmatically), the variable falls back to a type-appropriate default:

Type	Default when <code>def</code> is absent
<code>string</code>	<code>""</code>
<code>long_string</code>	<code>""</code>
<code>number</code>	<code>0</code>
<code>boolean</code>	<code>false</code>
<code>list</code>	<code>[]</code>
<code>option_single</code>	the first entry in <code>opts</code>
<code>option_multi</code>	<code>[]</code>
<code>object</code>	an object whose each field is set to its own default per this table (recursively)
<code>object_shape</code>	not asked at its definition site; its <code>def</code> /field defaults are applied where it is referenced (see §3.4)

These fallbacks are also used to synthesize missing nested fields when rendering with defaults.

3.1 Supported Variable Types

All type names are **lowercase**.

Type	Description	Requires <code>etype</code>	Requires <code>ofields</code>	Requires <code>opts</code>
<code>string</code>	Plain string	No	No	No
<code>long_string</code>	Multi-line or long string (e.g. code)	No	No	No
<code>number</code>	Floating-point number	No	No	No
<code>boolean</code>	<code>true</code> / <code>false</code>	No	No	No

Type	Description	Requires etype	Requires ofields	Requires opts
list	List of free entered values	Yes	No	No
object	Struct-like object with named fields. Asked to the user as a parameter in declaration order.	No	Yes (or type: <object_shape>)	No
object_shape	Reusable object shape (same fields as object). Not asked to the user at its definition site; only asked where it is referenced.	No	Yes	No
option_single	Single choice from a list of options	Yes (optional, defaults to string)	No	Yes (≥ 2)
option_multi	Multiple choices from a list of options	Yes	No	Yes (≥ 2)

A `long_string` variable also accepts a heredoc form for `def` (see §3.5).

The `etype` of an `option_single` / `option_multi` may be `string`, `long_string`, `number`, or the name of a declared `object_shape` variable (§3.4). `boolean` is not a valid option etype. When `etype` is a referenced `object_shape`, the `label` field is **required** (§3.6).

`object_shape` and `object` both describe an object shape via `ofields`, but differ in how they are presented to the user at build time: an `object` is a **collectible parameter** (the builder prompts for its fields in declaration order), while an `object_shape` is a **pure type definition** and is never prompted for on its own — it is only collected at the site that references it (a `list` / `option_*` element, or an `object` inheriting its fields via `type: <name>`). Both `object` and `object_shape` may appear as nested field types; when nested, both are collected inline during their parent's collection (the distinction only matters at root level).

3.2 Nested Objects

An `object` variable is a **collectible parameter**: at build time the builder prompts the user for each of its fields, in declaration order (interleaved with the other top-level params in the order they were declared). Its fields are declared under an `ofields` block. Field definitions may themselves be `object` variables, allowing arbitrary recursion depth.

```
params:
  server:
    type: object
    ofields:
      host:
        type: string
        def: "localhost"
      port:
        type: number
        def: 8080
      ssl:
        type: object
        ofields:
          enabled:
            type: boolean
            def: true
          cert:
            type: string
            def: ""
```

An `ofields` entry of an `object` (or `object_shape`) may **not** reference a top-level `object` param: there is no `etype: <object_name>` form and no `type: <object_name>` form. To reuse a shape inside an `object` (or `object_shape`) field, declare the shared shape as an `object_shape` and use it via `type: <name>` (§3.4). Nesting of `object` / `object_shape` fields inside `object` / `object_shape` `ofields` remains fully supported (as in the `ssl` field above).

3.3 Validation Rules

- `type` must be one of the values listed in §3.1.
- `etype` is only allowed when `type` is `list`, `option_single`, or `option_multi`. It is not allowed on `object` or `object_shape` (an object's shape is described by `ofields`).
- `etype` may be either a built-in type name (§3.1) — including the literal `object`, in which case the variable declares its element shape inline via its own `ofields` — or the name of a declared `object_shape` variable (see §3.4). Naming a declared `object` variable (instead of an `object_shape`) via `etype` is an error — only `object_shape` is referenceable by name.
- For `list`, `etype` may only be `string`, `long_string`, `number`, `boolean`, the inline `object` (with its own `ofields`), or a referenced `object_shape`. `list`, `option_single`, `option_multi`, and `object_shape` (the literal type name) are not valid list etypes.
- For `option_single` and `option_multi`, `etype` may only be `string`, `long_string`, `number`, the inline `object` (with its own `ofields`), or a referenced `object_shape`. `boolean`, `list`, `option_single`, `option_multi`, and `object_shape` (the literal type name) are not valid option etypes.
- For `option_single`, `etype` is optional and defaults to `string` when omitted. For `option_multi`, `etype` is required.
- `label` is only allowed when `type` is `option_single` or `option_multi`, and is **required** when their `etype` is a referenced `object_shape` (the by-name form). `label` MUST name a field declared on the referenced `object_shape`, and that field's type MUST be `string` or `long_string`. `label` is ignored for scalar etypes and for the inline `object` `etype`.
- An element reference (the by-name form) MUST resolve to an `object_shape` variable that declares an `ofields` block. Element reference cycles are not allowed and MUST be reported as an error.
- `ofields` is only allowed when `type` is `object` or `object_shape`. On an `object`, `ofields` is the inline field map; on an `object_shape`, `ofields` is always a map of field definitions (a shape declaration). An `object` must declare either `ofields` (inline) or `type: <object_shape_name>` (reuse a shape), but not both — `type: <name>` and `ofields` are mutually exclusive.
- `type` accepts either a built-in type name (§3.1) or the name of a declared `object_shape` variable (§3.4.2). A non-builtin `type` value names a declared `object_shape` whose `ofields` are spliced in as this `object`'s own fields; the variable is then an `object` value of that shape (asked at root, collected inline when nested). Naming a declared `object` (instead of an `object_shape`) via `type` is a parse error — only `object_shape` is referenceable by name. Built-in type names are reserved and cannot be used as `object_shape` names.
- An `ofields` entry of an `object` (or `object_shape`) may not reference a top-level `object` param: the only reusable-shape reference forms are `type: <object_shape_name>` (on `object` / `object_shape` fields) and `etype: <object_shape_name>` (on `list` / `option_*`). Referencing an `object` via `type` or `etype` is a parse error.
- `opts` is only allowed for `option_single` and `option_multi`. Any other type declaring `opts` is a parse error. `opts` MUST contain **at least two entries** — a single-option menu is not meaningful and is a parse error.
- All values supplied via `def`, `opts`, etc. must match the declared `type` and `etype`. Specifically, each entry in `opts` MUST be coercible to the option's `etype`: a `string` / `long_string` entry for a `string` / `long_string` `etype`, a `number` entry for a `number` `etype`, and an object literal matching the referenced `object_shape`'s `ofields` shape (or the inline `ofields` shape) for an `object` / `object_shape` `etype`. A mismatch is a parse error.
- The `def` value MUST match the declared `type` (and, for `list` / `option_multi`, its `etype`): a `def` for a `number` variable must be a number literal; a `def` for a `boolean` must be `true` / `false`; a `def` for a `string` / `long_string` must be a string; a `def` for an `object` or `object_shape` must be an object literal; a `def` for a `list` must be a list whose every element matches `etype`; a `def` for an `option_single` must be a single value matching `etype`; a `def` for an `option_multi` must be a list whose every element matches `etype`. A `def` value of the wrong kind is a parse error. (Object `def`s are checked for kind only — `object` — not for full field-shape conformance; extra or missing nested keys are tolerated and filled from defaults at render time.)

3.3.1 Literal Value Syntax

Values supplied for `def` and `opts` may be written in any of the following forms:

- **Quoted string** — `"..."` or `'...'`. Both styles yield the same value; a quote character that does not start a string literal must be enclosed in the other quote style. Used for `string`, `long_string`, and string-valued `option_*` entries.
- **Bare number** — a floating-point literal such as `80`, `3.14`, or `-1`. Used for `number` and number-valued entries.
- **Bare boolean** — `true` or `false`.
- **Inline list** — `[v1, v2, ...]`, items separated by commas. Items may be any of the forms in this section, including nested lists/objects. Whitespace around items is ignored.
- **Inline object** — `{ key: value, key2: value2, ... }`. Keys are bare identifiers; values are any of the forms in this section. Commas separate entries; nested `{}` / `[]` and string literals are respected, so values may themselves contain commas or colons.
- **Block list** — instead of an inline `[...]`, a `def:` (or `opts:`) line with an empty value may be followed by indented `<value>` lines (at indent + 4 spaces). Each item is parsed with the rules above.
- **Heredoc** — for `long_string` `def` only, see §3.5.

Bare (unquoted) tokens that are not `true` / `false` / numbers / inline collections are treated as strings.

3.4 Object Type Reuse (Element References)

The `etype` of a `list`, `option_single`, or `option_multi` may name a **previously- or forward-declared** `object_shape` **variable** instead of a built-in type. The referenced `object_shape`'s `ofields` shape is then reused as the element structure of the list/options, so the field definitions need not be repeated inline. Likewise, an `object` (or a nested object field) may reuse a declared `object_shape`'s `ofields` by writing `type: <object_shape_name>` instead of `type: object` + an inline `ofields` map; the referenced `object_shape`'s fields are spliced in as the object's own fields.

The referenced variable **MUST** be declared in the same `params` block with `type: object_shape` and an `ofields` block. References are resolved at parse time; after resolution the referencing variable behaves exactly as if the referenced `object_shape`'s `ofields` had been written inline. Forward references (declaring the referencing variable before the `object_shape` it names) are permitted. Circular references are not allowed and **MUST** be reported as errors. For `option_single` / `option_multi` with an `object_shape` `etype`, the `label` field (§3.6) is required.

An `object_shape` is **not** asked to the user at its definition site — it is a pure type definition. It is only collected at the site that references it: a `list` / `option_*` element prompt, or an `object` / field that uses it via `type: <name>` (in which case that `object` / field is the one prompted, with the shape's fields). A top-level `object` (not `object_shape`) cannot be referenced via `etype` / `type`; only `object_shape` is referenceable. Both `object` and `object_shape` may appear as nested field types; when nested they are collected inline during parent collection (identically), so the `object`-vs-`object_shape` distinction only matters at root level.

3.4.1 etype reference (list / option_single / option_multi)

```
params:
  host:
    type: object_shape
    ofields:
      host:
        type: string
        def: "localhost"
      port:
        type: number
        def: 8080
  servers:
    type: list
    etype: host
    def:
      - { host: "localhost", port: 8080 }
      - { host: "db.local", port: 5432 }
```

```
--
Hosts:
{{{for S in SERVERS}}}
- [[S.HOST]]: [[S.PORT]]
{{{end for}}}
```

renders to:

```
Hosts:
- localhost:8080
- db.local:5432
```

The same mechanism works for `option_multi`, where each chosen option is an object shaped like the referenced `object_shape`:

```
params:
  feature:
    type: object_shape
    ofields:
      name:
        type: string
      enabled:
        type: boolean
        def: false
    selected:
      type: option_multi
      etype: feature
      label: name
      opts:
        - { name: "auth", enabled: true }
        - { name: "logs", enabled: false }
--
Enabled:
{{{for F in SELECTED}}}
- [[F.NAME]]
{{{end for}}}
```

3.4.2 Shape reuse (`type: <object_shape_name>`)

An `object` (or a nested object field) may reuse the fields of a declared `object_shape` by writing `type: <object_shape_name>` instead of `type: object` with an inline `ofields` map. The resulting `object` is still a **collectible parameter** (it is prompted at build time), but its field shape is the referenced `object_shape`'s `ofields`. This is the form to use when an object param (or field) should reuse a shared shape defined once. `type: <name>` is mutually exclusive with `ofields` (the referenced `object_shape` provides the fields). Built-in type names are reserved and cannot shadow or be shadowed by an `object_shape` name.

```
params:
  host:
    type: object_shape
    ofields:
      host:
        type: string
        def: "localhost"
      port:
        type: number
        def: 8080
  cfg:
    type: host
```

```
--
Host: [[[CFG.HOST]]] Port: [[[CFG.PORT]]]
```

Renders (with defaults) to:

```
Host: localhost Port: 8080
```

At build time the builder prompts for `cfg` (not `host`), collecting `host` and `port` as its fields. The `host` object_shape itself is never prompted for on its own.

3.5 Long String Defaults (Heredoc Form)

For `long_string` variables, the `def` value may alternatively be supplied as a **heredoc block**. This is convenient when the default is a long, multi-line text such as a code snippet or a paragraph that is unpleasant to write as a single quoted line with embedded escapes.

The syntax is:

```
def: >>>
raw content lines,
at any indentation
<<<
```

- The value of the `def:` key is the literal token `>>>` (with nothing else on the line).
- Every subsequent line — regardless of indentation — is part of the default value, taken **verbatim** (no escape processing, no quote stripping).
- The block ends at the first line whose trimmed content is exactly `<<<`. That terminator line is consumed and is not part of the value.
- If the end of file is reached before a `<<<` terminator, parsing fails with an error.
- The heredoc form is **only** valid for `long_string` variables. Using it on any other type is a parse error.

The collected lines are joined with `\n` to form the default value; the newline that precedes the `<<<` terminator is not included, so the value is exactly the text between `>>>` and `<<<`.

```
params:
  body:
    type: long_string
    desc: Default JSON request body
    def: >>>
{
  "name": "example",
  "active": true
}
<<<
--
POST [[[URL]]]
Content-Type: application/json

[[[BODY]]]
```

renders (with default) to:

```
POST https://api.example.com
Content-Type: application/json
```

```
{
  "name": "example",
  "active": true
}
```

3.6 Option Labels (`label`)

When an `option_single` or `option_multi` variable has an `object_shape` etype, each entry in `opts` is an object literal. The interactive menu needs a single, human-readable string to display for each option, so a `label` field is **required** in that case. `label` names a field declared on the referenced `object_shape`; the value of that field on each option object becomes its menu label.

The named field **MUST** be declared on the referenced `object_shape` with type `string` or `long_string`. `label` is only meaningful for `option_single` / `option_multi` with an `object_shape` etype and is ignored (and should be omitted) for scalar etypes.

```
params:
  feature:
    type: object_shape
    ofields:
      name:
        type: string
      enabled:
        type: boolean
        def: false
    selected:
      type: option_multi
      etype: feature
      label: name
      opts:
        - { name: "auth", enabled: true }
        - { name: "logs", enabled: false }
  --
Enabled:
{{{for F in SELECTED}}}
- [[F.NAME]]
{{{end for}}}
```

In the menu, the two options are shown as `auth` and `logs` (the values of the `name` field). The chosen objects are stored whole, so `[[F.NAME]]` and `[[F.ENABLED]]` resolve normally inside a loop.

4. Prompt Body Syntax

The prompt body is free text that may contain three kinds of dynamic constructs:

4.1 Variable Placeholders

A variable reference is written as `[[VAR_NAME]]`. Variable references **MUST** be written in **uppercase**. The matching against the variable name declared in `params` (which stays lowercase) is case-insensitive, so a variable declared as `username` is referenced as `[[USERNAME]]`. Using a lowercase or mixed-case reference (e.g. `[[name]]`, `[[UserName]]`) is a parse error.

```
Hello, [[USERNAME]]!
```

4.1.1 List

A `list` variable referenced as `[[[VAR]]]` renders its elements joined with `", "`. The rendering of each element depends on its `etype`:

- For scalar `etype`s (`string` , `long_string` , `number` , `boolean`), each element is rendered as its literal value (see §4.6 for value rendering rules).
- For an `object` `etype` (declared inline with its own `ofields` , or via a referenced `object_shape` per §3.4), each element is rendered as a comma-separated list of `field: value` pairs (no enclosing braces), e.g. `host: localhost, port: 8080` . For structured output, prefer dotted-path access (§4.1.2), field projection (§4.1.5), or a `for` loop (§4.3).

```
params:
  tags:
    type: list
    etype: string
    def: ["api", "v2", "beta"]
  ports:
    type: list
    etype: number
    def: [80, 443, 8080]
  flags:
    type: list
    etype: boolean
    def: [true, false, true]
  servers:
    type: list
    etype: object
    ofields:
      host:
        type: string
        def: "localhost"
      port:
        type: number
        def: 8080
    def:
      - { host: "localhost", port: 8080 }
      - { host: "db.local", port: 5432 }
--
Tags:  [[[TAGS]]]
Ports: [[[PORTS]]]
Flags: [[[FLAGS]]]
Servers: [[[SERVERS]]]
```

renders to:

```
Tags:  api, v2, beta
Ports: 80, 443, 8080
Flags: true, false, true
Servers: host: localhost, port: 8080, host: db.local, port: 5432
```

For structured element access, iterate the list with a `for` loop (see §4.3) and use a dotted path on the loop variable to print a single field:

```
Hosts:
{{{for SERVER in SERVERS}}}
- [[[SERVER.HOST]]]
{{{end for}}}
```

renders to:

```
Hosts:
- localhost
- db.local
```

4.1.2 Object Field Access (Dotted Paths)

Fields of an `object` variable are referenced with a **dotted path** inside the placeholder: `[[[<OBJECT>.<FIELD>]]]`. Paths may chain through any number of nested objects, so a field declared at arbitrary depth is reached as `[[[OBJ1.OBJ2.OBJ3.FIELDNAME]]]`. Every path segment **MUST** be uppercase; segments resolve case-insensitively against the `ofields` declared in `params` (which stay lowercase).

```
params:
  server:
    type: object
    ofields:
      host:
        type: string
        def: "localhost"
      ssl:
        type: object
        ofields:
          enabled:
            type: boolean
            def: true
--
Host: [[SERVER.HOST]]
SSL: [[SERVER.SSL.ENABLED]]
```

Dotted paths also work inside loop bodies, where the leading segment is the loop variable (see §4.3) bound to the current list element. The loop variable **MUST** be uppercase:

```
{{for ENDPOINT in ENDPOINTS}}
- [[[ENDPOINT.METHOD]]] [[[ENDPOINT.PATH]]]
{{end for}}
```

4.1.3 Option Single

An `option_single` variable holds exactly one value chosen from its `opts`. The optional `etype` (default `string`) determines the kind of each option; the supported etypes are `string`, `long_string`, `number`, the inline `object`, and a referenced `object_shape` (§3.4, requires `label` — §3.6). Referencing the variable as `[[[VAR]]]` renders the selected option's value: scalars render verbatim (see §4.6), and an object option renders as a comma-separated `field: value` list with no enclosing braces (use dotted-path access, §4.1.2, to extract fields). When no value is supplied, the `def` default is used.

```
params:
  env:
    type: option_single
    desc: Target environment
    opts:
      - "development"
      - "staging"
      - "production"
    def: "production"
--
Deploying to [[[ENV]]].
```

renders (with default) to:

```
Deploying to production.
```

A number-etype example:

```
params:
  port:
    type: option_single
    etype: number
    opts:
      - 80
      - 443
      - 8080
    def: 443
  --
  Listening on port [[[PORT]]].
```

renders (with default) to:

```
Listening on port 443.
```

4.1.4 Option Multi

An `option_multi` variable holds zero or more values chosen from its `opts`. The required `etype` determines the kind of each option; the supported etypes are `string`, `long_string`, `number`, the inline `object`, and a referenced `object_shape` (§3.4, requires `label` — §3.6). Referencing the variable as `[[[VAR]]]` renders the selected values joined with `" , "`, in the order they were chosen. An empty selection renders as an empty string. When `etype` is `object` (inline or via an `object_shape`), each chosen option is an object; use dotted-path access inside a loop or projection to extract fields.

```
params:
  features:
    type: option_multi
    desc: Feature flags to enable
    opts:
      - "auth"
      - "logs"
      - "metrics"
      - "cache"
    def: ["auth", "metrics"]
  --
  Enabled features: [[[FEATURES]]]
```

renders (with default) to:

```
Enabled features: auth, metrics
```

4.1.5 List Field Projection

When a dotted path traverses into a `list` of `object` elements, the next segment is **projected** across every element of the list. The result is the list of that field's values from each element, joined with `" , "`. This lets a single placeholder expand a whole column of a list-of-objects without a loop:

```

params:
  model:
    type: object
    ofields:
      fields:
        type: list
        etype: object
        ofields:
          name:
            type: string
        def:
          - { name: "username" }
          - { name: "email" }
  --
const { [[MODEL.FIELDS.NAME]] } = req.body;

```

renders (with default) to:

```
const { username, email } = req.body;
```

4.2 Conditional Expressions (Ternary)

A conditional is written as `{{condition ? value_if_true : value_if_false}}`.

```
{{[[AGE]] >= 18 ? "adult" : "minor"}}
```

Conditions may reference variables (`[[VAR]]` , uppercase) or literal values, and may use any of the operators in §5. String literals may be written with either double (`"..."`) or single (`'...'`) quotes; both yield the same value. A quote character that does not start a string literal must be escaped by enclosing the literal in the other quote style.

4.3 For Loops

A loop iterates over a list-valued variable — i.e. a `list` variable or an `option_multi` variable — and repeats its body once per element.

```

{{{for ENDPOINT in ENDPOINTS}}}
- [[ENDPOINT.METHOD]] [[ENDPOINT.PATH]]
{{{end for}}}

```

- `{{{for <ITEM> in <LIST>}}}` opens the loop. `<ITEM>` is the loop variable binding and MUST be uppercase; `<LIST>` is a bare variable reference (also uppercase) — it is written without `[[...]]` wrapping, since `[[...]]` is reserved for printing values into the prompt body. For backward compatibility, `[[VAR]]` wrapping is still tolerated and stripped.
- `{{{end for}}}` closes the loop.
- Inside the body, the current item is referenced by `<ITEM>.<FIELD>` (both segments uppercase).

4.4 If Blocks (Conditional Blocks)

A conditional block renders its content only when the condition is **truthy** (see §4.6.2 for the truthiness rules). The condition variable MUST be uppercase.

```

{{{if INCLUDE_AUTH}}}
Authorization: [[ENDPOINT.HEADERS.AUTHORIZATION]]

```

```
{{{end if}}}
```

4.5 Escaping and Safety

Only `[[[` and `{{{` trigger expansion. An opening `[[[` without a matching closing `]]]`, or a bare `{{{` without a matching `}}}`, is emitted verbatim, so code snippets containing similar-looking sequences are not misinterpreted. However, recognized block constructs — `{{{for ...}}}`, `{{{end for}}}`, `{{{if ...}}}`, `{{{end if}}}` — MUST be balanced: an unclosed `for` / `if`, or a stray `{{{end for}}}` / `{{{end if}}}` with no matching opener, is a parse error.

4.6 Value Rendering and Truthiness

4.6.1 Value Rendering

When a value is substituted into the prompt body (via `[[[VAR]]]`, a ternary branch, or list joining), it is rendered as a string according to its type:

Type	Rendering
<code>string</code>	The string verbatim.
<code>long_string</code>	The string verbatim (may contain newlines).
<code>number</code>	Integer-valued numbers render without a fractional part (e.g. <code>80</code>); otherwise the full floating-point representation is used (e.g. <code>3.14</code>). Negative numbers are prefixed with <code>-</code> .
<code>boolean</code>	<code>true</code> or <code>false</code> .
<code>list</code>	Elements rendered per this table, joined with <code>" , "</code> .
<code>object</code>	<code>field: value</code> pairs (each <code>value</code> rendered per this table), joined with <code>" , "</code> , without enclosing braces. Field order follows declaration order.

4.6.2 Truthiness

Conditions in `if` blocks, ternaries, and the operand of `!` are evaluated for truthiness as follows:

Type	Truthy when
<code>boolean</code>	the value is <code>true</code>
<code>string</code> / <code>long_string</code>	the string is non-empty
<code>number</code>	the value is non-zero
<code>list</code>	the list is non-empty
<code>object</code>	the object has at least one field

A bare variable reference used as a condition (e.g. `{{{if FLAG}}}`) is truthy per the table above.

5. Condition Operators

All operators are **left-associative** (the ternary `? :` is right-associative — see §5.1). Type checking is enforced at runtime; for example, comparing a number to a string fails evaluation. String literals may use either single (`'...'`) or double (`"..."`)

quotes interchangeably.

Operator	Meaning	Example
<code>=</code>	Equal (<code>==</code> is accepted as an alias)	<code>[[[A]]] = [[[B]]]</code>
<code>!=</code>	Not equal	<code>[[[X]]] != 'test'</code>
<code>!</code>	Logical NOT (unary)	<code>![[[FLAG]]]</code>
<code>contains</code>	String contains / list membership	<code>[[[TEXT]]] contains "hello" ; [[[TAGS]]] contains "api"</code>
<code>starts_with</code>	String starts with	<code>[[[PATH]]] starts_with "/home"</code>
<code>ends_with</code>	String ends with	<code>[[[EXT]]] ends_with ".js"</code>
<code>>=</code>	Greater or equal (numbers)	<code>[[[COUNT]]] >= 5</code>
<code>></code>	Greater than (numbers)	<code>[[[AGE]]] > 18</code>
<code><=</code>	Less or equal (numbers)	<code>[[[SCORE]]] <= 100</code>
<code><</code>	Less than (numbers)	<code>[[[PRICE]]] < 10</code>

Notes:

- `==` is tokenized as `=` ; the two are interchangeable.
- `contains` is overloaded: when either operand is a `list` , it performs membership testing (whether the list contains the other operand, compared by value); otherwise it tests whether the left string contains the right string. `starts_with` and `ends_with` apply only to strings.
- Comparison operators (`>` , `<` , `>=` , `<=`) require both operands to be numbers; a type mismatch fails evaluation.
- `=` and `!=` require operands of the same scalar kind (number/number, string/string-or-long_string, boolean/boolean); a mismatch fails evaluation.
- The string operators (`contains` , `starts_with` , `ends_with`) may also be written in method-call form: `[[[VAR]]].contains("x")` , `[[[VAR]]].starts_with("x")` , `[[[VAR]]].ends_with("x")` . This form is rewritten to the infix form before evaluation and is equivalent.

5.1 Operator Precedence

From highest to lowest:

- `!` (unary NOT)
- `>` , `<` , `>=` , `<=`
- `=` , `!=`
- `contains` , `starts_with` , `ends_with`
- `? :` ternary (right-associative)

The ternary is the lowest-precedence operator. Parentheses may be used to group sub-expressions. Note: branches of a ternary are plain values (a `[[[VAR]]]` reference, a quoted string, a bare number/boolean, or literal text); **nested ternaries are not supported** inside ternary branches.

6. Features

Feature	Supported
String & long string variables	Yes
Number, boolean, list, option_single, option_multi	Yes
Nested <code>object</code> variables (with <code>ofields</code> block)	Yes
<code>object_shape</code> type definitions (reusable shape, not asked)	Yes
Variable defaults (<code>def</code>)	Yes
Option lists (<code>opts</code>)	Yes
Nested <code>etype</code> for lists / option_multi	Yes
Object type reuse via <code>etype: <object_shape></code> references (§3.4)	Yes
Shape reuse via <code>type: <object_shape_name></code> on <code>object</code> /fields (§3.4.2)	Yes
Inline <code>etype: object</code> (with inline <code>ofields</code>)	Yes
<code>[[[VAR]]]</code> placeholder syntax	Yes
<code>[[[OBJ.FIELD]]]</code> dotted-path object field access	Yes
<code>[[[OBJ1.OBJ2.OBJ3.FIELD]]]</code> arbitrary-depth nesting	Yes
<code>[[[LIST.FIELD]]]</code> list-of-objects field projection	Yes
<code>{{{cond ? a : b}}}</code> ternary condition	Yes
All operators listed in §5 (incl. <code>==</code> alias, method-call form)	Yes
<code>{{{for <ITEM> in <LIST>}}}<content>{{{end for}}}</code> loops	Yes
<code>{{{if <COND>}}}<content>{{{end if}}}</code> conditional blocks	Yes
Complex conditionals with variables and literals	Yes
Runtime type checking during evaluation	Yes
Code snippets containing <code>[[[...]]</code> sequences	Yes
Recursive object definitions (objects inside objects)	Yes
<code>source</code> provenance metadata field (§2.1)	Yes

7. Data Model (Conceptual)

The following describes the conceptual structure of a parsed UPL document. Field names mirror the metadata keys in the file.

7.1 Prompt

Field	Type	Description
<code>name</code>	string	The prompt identifier. MUST match the file's base name and be lowercase alphanumeric (UTF-8) + underscores only (see §2).
<code>title</code>	string (optional)	Human-readable title.
<code>desc</code>	string (optional)	Description text.
<code>source</code>	string (optional)	Provenance <code><host>/<username>/<prompt_name></code> for prompts pulled from a repository (§2.1). Absent for locally authored prompts.
<code>prompt</code>	string	The raw prompt body (the source text before placeholder substitution). Rendering is performed separately by the builder.
<code>variable_definitions</code>	map	Declared input variables.
<code>variable_defaults</code>	map	Default values keyed by variable name (dotted path for nested fields).
<code>template</code>	Template	Parsed body AST (see §7.6). Not serialized; reconstructed from <code>prompt</code> on load.

7.2 VariableDefinition

Field	Type	Description
<code>type</code>	VariableType	One of the types in §3.1.
<code>desc</code>	string (optional)	Description.
<code>options</code>	list (optional)	Allowed options (for <code>option_single</code> / <code>option_multi</code>). Each entry matches <code>element_type</code> .
<code>element_type</code>	VariableType (optional)	Element type (for <code>list</code> / <code>option_single</code> / <code>option_multi</code>). When <code>etype</code> names a declared <code>object_shape</code> variable (§3.4), this is <code>object</code> and <code>ofields_definitions</code> holds the referenced <code>object_shape</code> 's resolved fields.
<code>element_ref</code>	string (optional)	Name of the declared <code>object_shape</code> variable referenced via <code>etype</code> : <code><name></code> (resolved at parse time; kept for downstream default synthesis).
<code>label</code>	string (optional)	For <code>option_single</code> / <code>option_multi</code> with an <code>object_shape</code> <code>etype</code> : the field whose value is the menu label (§3.6).
<code>type_ref</code>	string (optional)	Name of the declared <code>object_shape</code> variable whose <code>ofields</code> an <code>object</code> (or nested object field) reuses via <code>type</code> : <code><name></code> (resolved at parse time into <code>ofields_definitions</code>); <code>type</code> is then <code>Object</code> .
<code>ofields_definitions</code>	map (optional)	Object fields (for <code>object</code> / <code>object_shape</code> , or resolved from an <code>object_shape</code> reference).

7.3 Value

A value is one of:

- **String** — plain string.
- **LongString** — multi-line string. (Rendered identically to String; the distinction is preserved so authoring tools can offer multi-line input.)
- **Number** — floating-point.
- **Boolean** — true/false.
- **List** — ordered list of values.
- **Object** — ordered map of string → value. Iteration and rendering preserve field declaration order.

7.4 Conditional Expression

A condition is represented as an expression tree:

- **Binary** — `left op right` (operators per §5)
- **Unary** — `op expr` (e.g. `!flag`)
- **Literal** — a constant value.
- **Variable** — a reference to a named variable.

7.5 ForLoop

Field	Type	Description
<code>item_name</code>	string	Name of the loop variable.
<code>list_variable</code>	string	Name of the list-valued variable being iterated (a <code>list</code> or <code>option_multi</code>).
<code>body</code>	list	Body nodes rendered once per element (see §7.6).

7.6 Template / Node

The parsed body is a tree of nodes (`Template { nodes: Vec<Node> }`). A node is one of:

- **Text(string)** — literal text emitted verbatim.
- **Placeholder(string)** — a `[[[VAR]]]` or `[[[OBJ.FIELD]]]` reference, resolved and rendered per §4.1 / §4.6.
- **Ternary { cond, true_branch, false_branch }** — `{{{cond ? a : b}}}`; `cond` is a Conditional Expression (§7.4); branches are strings rendered per §4.6.
- **Loop { item, list, body }** — a `for` loop (§4.3); `body` is a `Vec<Node>`.
- **If { cond, body }** — an `if` block (§4.4); `body` is a `Vec<Node>`.

8. Examples

8.1 Ask for a REST Client

```
--
name: ask_rest_client
title: Ask for a REST Client from an Endpoint List
desc: Ask the assistant to write a Node.js fetch client for the given endpoints
params:
  endpoints:
    type: list
    desc: List of REST endpoints the client should cover
    etype: object
    ofields:
      method:
```

```

    type: option_single
    desc: HTTP method
    opts:
      - "GET"
      - "POST"
      - "PUT"
      - "DELETE"
    def: "GET"
  path:
    type: string
    desc: Endpoint path
    def: "/api/users"
  body:
    type: long_string
    desc: Request body (if any)
    def: "{}"
  headers:
    type: object
    desc: Custom headers
    ofields:
      content-type:
        type: string
        def: "application/json"
      authorization:
        type: string
        def: ""
    def: {}
  include_auth:
    type: boolean
    desc: Whether an auth header should be required
    def: true
--
Please write a Node.js client that calls the following endpoints using fetch:

{{{for ENDPOINT in ENDPOINTS}}}
- [[[ENDPOINT.METHOD]]] [[[ENDPOINT.PATH]]] (body: [[[ENDPOINT.BODY]]])
{{{if INCLUDE_AUTH}}}
  Note: this endpoint must send an Authorization header.
{{{end if}}}
{{{if ENDPOINT.BODY != "{}"}}}
  Note: this endpoint expects a request body.
{{{end if}}}
{{{end for}}}

Explain how the client should handle errors and retries for each call.
--

```

8.2 Ask for File Handling Advice

```

--
name: ask_file_handling
title: Ask for File Handling Advice
desc: Ask the assistant how to handle a file based on its type and size
params:
  file_type:
    type: option_single
    desc: Type of file (JS, JSON, HTML)
    opts:
      - "js"
      - "json"
      - "html"
    def: "js"
  file_size:
    type: number
    desc: Size in KB
    def: 100
  use_async:
    type: boolean

```

```

    desc: Whether the caller prefers async/await
    def: true
--
I have a [[[FILE_TYPE]]] file of roughly [[[FILE_SIZE]]] KB.

{{{FILE_SIZE > 100 ? "It is a large file, so memory usage matters." : "It is a small file, so simplicity matters

{{{USE_ASYNC ? "Please recommend an async/await approach." : "Please recommend a synchronous approach."}}}

Describe the best way to read and process this file in Node.js, and explain why.
--

```

8.3 Ask for an API Configuration Review

```

--
name: ask_api_config_review
title: Ask for an API Configuration Review
desc: Ask the assistant to review and explain an API client configuration
params:
  api_config:
    type: object
    desc: Full API configuration to review
    ofields:
      base_url:
        type: string
        desc: Base URL of the API
        def: "https://api.example.com"
      timeout:
        type: number
        desc: Request timeout in seconds
        def: 30
      auth:
        type: object
        desc: Authentication config
        ofields:
          type:
            type: option_single
            opts:
              - "bearer"
              - "basic"
              - "none"
            def: "bearer"
          token:
            type: string
            desc: Bearer token
            def: "my-secret-token"
          username:
            type: string
            def: "admin"
          password:
            type: string
            def: "secret"
      retry:
        type: object
        desc: Retry strategy
        ofields:
          max_attempts:
            type: number
            def: 3
          delay_ms:
            type: number
            def: 1000
        def: { max_attempts: 3, delay_ms: 1000 }
--
Please review the following API configuration and tell me whether it is safe and sensible:

- Base URL: [[[API_CONFIG.BASE_URL]]]
- Timeout (seconds): [[[API_CONFIG.TIMEOUT]]]

```

```

- Auth type: [[[API_CONFIG.AUTH.TYPE]]]
- Auth username: [[[API_CONFIG.AUTH.USERNAME]]]
- Retry max attempts: [[[API_CONFIG.RETRY.MAX_ATTEMPTS]]]
- Retry delay (ms): [[[API_CONFIG.RETRY.DELAY_MS]]]

```

```

Point out any security issues (for example, hardcoded credentials) and suggest improvements.
--

```

8.4 Ask for a Server Inventory (object_shape reuse)

This example shows the difference between `object` (asked to the user) and `object_shape` (a reusable shape, never asked on its own). `host` is an `object_shape` reused by the `servers` list and by the `primary` object (via `type: host`).

```

--
name: ask_server_inventory
title: Ask for a Server Inventory
desc: Collect a list of servers plus a primary server, all sharing one shape
params:
  host:
    type: object_shape
    ofields:
      name:
        type: string
        def: "localhost"
      port:
        type: number
        def: 8080
  servers:
    type: list
    etype: host
    def:
      - { name: "web", port: 80 }
      - { name: "db", port: 5432 }
  primary:
    type: host
    def: { name: "web", port: 80 }
--
Primary: [[[PRIMARY.NAME]]]:[[[PRIMARY.PORT]]]
All:
{{{for S in SERVERS}}}
- [[[S.NAME]]]:[[[S.PORT]]]
{{{end for}}}
--

```

At build time the builder prompts for `servers` (a list whose each item is collected using the `host` `object_shape` shape) and for `primary` (an object reusing the `host` shape via `type: host`). It never prompts for `host` on its own.

9. Parsing and Validation

A conforming UPL implementation MUST perform the following steps:

1. **Parse** — split the file into metadata and body sections using the `--` delimiter (the leading `--` is optional). Parse `def / opts` values per the literal syntax in §3.3.1 (including the `long_string` heredoc form, §3.5).
2. **Validate header** — ensure required metadata fields are present and well-formed. In particular, `name` is required, MUST contain only lowercase alphanumeric (UTF-8) characters and underscores, and MUST equal the file's base name (the file name with its `.txt / .upl` extension — and a single trailing `.prompt` segment, if present — stripped). The file MUST use the `.txt` or `.upl` extension. `source` (§2.1), if present, is accepted as informational metadata.
3. **Validate params** — ensure every variable declaration has a valid `type` and that the type-specific fields (`etype` , `ofields` , `opts` , `label`) are used only where permitted (§3.3) (note: `type: <object_shape_name>` is not a separate key

— it is the `type` value naming a declared `object_shape`). Resolve element references (§3.4) and verify they point to declared `object_shape` variables with `ofields`, reporting any cycle. Resolve `type: <object_shape_name>` references on `object` variables/fields. Reject any by-name `etype / type` reference that names a declared `object` (only `object_shape` is referenceable by name).

4. **Validate field types** — for each variable, ensure `def` and `opts` values match the declared `type` and `element_type` (§3.3). A `def` whose `variableValue` kind does not match the declared type (and, for lists, whose elements do not match `etype`) is a parse error.
5. **Validate body** — parse the body into a Template (§7.6); ensure every variable reference is written in uppercase (§4.1); ensure `for / if` blocks are balanced (§4.5). For each dotted-path reference whose root names a declared variable (or an in-scope loop variable), verify that every segment after the root names a real field of the referenced object's resolved shape (an unknown field is a parse error). A root variable that is not declared in `params` is **not** a parse error: its value may be supplied programmatically at render time (see the `[[URL]]` example in §3.5), and existence of a *value* is enforced at render time (step 6) as `MissingValue`.
6. **Render** — substitute variables, evaluate conditionals with runtime type checking (§5), and expand loops, using the rendering and truthiness rules in §4.6. A reference to a variable for which no value was supplied fails here with `MissingValue`; a `for` loop over a non-list value fails here as well.

Errors raised at any step SHOULD include the offending field name, line number, and a clear description of the failure.

10. Conformance

An implementation conforms to this standard if it:

- Parses files with the `.txt` or `.upl` extension in the format described in §2 (leading `--` optional).
- Enforces that the `name` metadata field is present, lowercase alphanumeric (UTF-8) plus underscores only, and matches the file's base name (a single trailing `.prompt` segment stripped if present).
- Accepts the optional `source` metadata field (§2.1) without affecting parsing/rendering.
- Supports all variable types in §3.1 with their associated validation rules (§3.3), including the literal value syntax in §3.3.1 and the `long_string` heredoc form (§3.5).
- Rejects `def` values whose kind does not match the declared `type / etype` (§3.3) as parse errors.
- Supports object type reuse via `etype: <object_shape>` references (§3.4) and the `label` field for `object_shape`-etype options (§3.6). Supports `type: <object_shape_name>` shape reuse on `object` variables/fields (§3.4.2). Supports the inline `etype: object` (with inline `ofields`). Treats `object` `params` as collectible (asked at build time) and `object_shape` `params` as pure type definitions (never asked at their definition site, only at reference sites).
- Expands `[[[VAR]]]` placeholders (including dotted-path access and list field projection), `{{{cond ? a : b}}}` ternaries, `{{{for ... in ...}}}{end for}}` loops (over `list` and `option_multi`), and `{{{if ...}}}{end if}}` blocks.
- Renders values and evaluates truthiness per §4.6. Object fields are rendered and iterated in **declaration order** (§4.6.1, §7.3).
- Validates, at parse time, that every dotted-path segment after a declared root names a real field of the referenced object's resolved shape (§9 step 5); an unknown field on a declared object is a parse error. A root variable not declared in `params` is allowed at parse time and resolved at render time (§9 step 6).
- Enforces runtime type checking for all operators in §5, including the `contains` list-membership overload and the `==` alias for `=`.
- Applies the operator precedence in §5.1.
- Reports `for / if` block imbalance as parse errors (§4.5).
- Reports errors as described in §9.